DATAMITES — *in association with Rubiex Internship Program*

ARTIFICIAL INTELLIGENCE CAPSTONE PROJECT

PRAICP-1002 : TrafSignDetc — Traffic Sign Recognition (Short Report)

Team Code	Project Code	Submitted By	Dataset	Report Date
PTID-AIE-JUL-26-11142	PRAICP-1002	Sameeksha	DataMites TrafSignDetc	October 2026

1. Problem Statement & Objective

Missing or misreading a traffic sign (stop, speed limit, warning) is a common cause of road accidents, and driver-assistance and self-driving systems must recognise signs automatically from camera images. This project builds a **Convolutional Neural Network (CNN)** that looks at a photo of a traffic sign and predicts which of **43 sign classes** it is. It covers the two goals of the DataMites brief: **(1) image preprocessing** and **(2) an ML model to predict the sign**, built as: explore data → preprocess and split → augment → baseline CNN → tune with Keras Tuner → evaluate → predict on a new image → save the model.

2. Dataset & Exploratory Analysis

The ZIP holds one folder per sign (folder name = label). It is too large to load fully, so it was **indexed without extracting**, and at most **400 random images per class** were used (seed 42).

Property	Full dataset	Working subset used
Images	39,209	15,460
Classes	43 (folders 0–42)	43
Smallest / largest class	0 (210) / 2 (2,250)	0 (210) / 400 (cap)
Imbalance ratio	10.71 : 1	1.90 : 1
Corrupt images / colour mode	—	0 / RGB (.png)
Original sizes	2,066 different sizes, 25×25 to 224×219 px (median ≈ 44×43)	

Table 1: Dataset summary.



Figure 1: Images per class in the full dataset (strongly imbalanced).

Key findings: classes are unequal; images are small and vary in brightness, blur, angle and size; and some signs look alike (for example speed limits differ only by the number). These findings justify resizing, class weights and augmentation.

3. Preprocessing, Split & Augmentation

1. **Resize** every image to RGB **64×64**; store as compact uint8 (190 MB instead of 760 MB).
2. **Normalise** by 1/255 using a Rescaling layer inside the model, so training and new images are treated identically (one shared preprocess_image function).

3. **Stratified split 70 / 15 / 15:** 10,822 train, 2,319 validation, 2,319 test. The test set is used only once, at the end.
4. **Class weights** (balanced, from training labels): 1.71 for the smallest classes (0, 19, 37) down to 0.90 for classes at the 400-image cap.
5. **Augmentation** (training only, built into the model): rotation ±11°, shift 10%, zoom 10%, contrast 20%. **No horizontal flip**, because mirror-image signs (turn left / turn right, keep left / keep right) would change class.

4. Models & Hyperparameter Tuning

A **baseline CNN** was built first: Rescaling → three Conv2D layers (32, 64, 128 filters, 3×3, each followed by 2×2 max-pooling) → Flatten → Dense(128) → Dropout(0.5) → Dense(43, softmax). It has **688,747 parameters** and uses the Adam optimiser, sparse categorical cross-entropy, batch size 32, up to 30 epochs, EarlyStopping and ReduceLROnPlateau.

Keras Tuner RandomSearch tried 8 of the 96 possible combinations (filters in layers 1 and 2, dense units, dropout, learning rate, batch size), ranked by **validation accuracy** only, taking about 2 h 26 min on CPU. A learning rate of 0.001 clearly beat 0.0001.

Rank	Batch	Filters 1	Filters 2	Dense	Dropout	LR	Val. Acc.
1 (best)	16	32	128	128	0.4	0.001	0.9918
2	32	32	64	256	0.3	0.001	0.9897
3	16	32	128	256	0.5	0.001	0.9871
8 (worst)	32	32	64	128	0.5	0.0001	0.9064

Table 2: Selected tuning trials (top 3 and worst of 8).

The **final CNN** was rebuilt with the best settings (conv filters 32/128/128, dense 128, dropout 0.4, learning rate 0.001, batch 16) and trained up to 40 epochs. It has **780,971 parameters**. Its validation accuracy rose from 49% (epoch 1) to above 99% with no sign of overfitting.

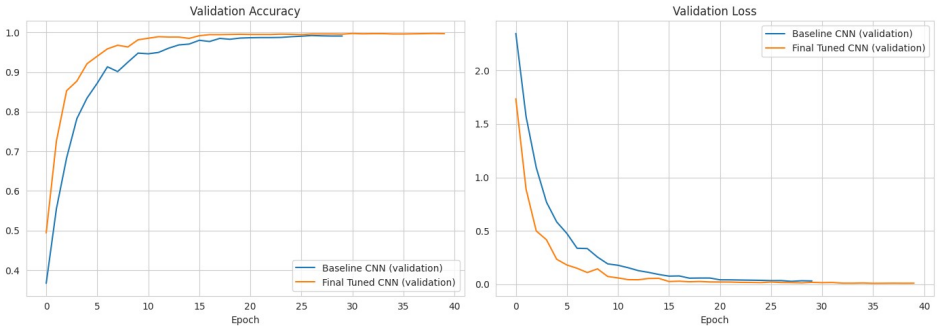


Figure 2: Validation accuracy and loss of the baseline and final models.

5. Results

Model / Split	Accuracy	Precision (macro)	Recall (macro)	F1 (macro)
Baseline CNN — Validation	0.9914	0.9920	0.9918	0.9918
Baseline CNN — Test	0.9935	0.9941	0.9939	0.9940
Final Tuned CNN — Validation	0.9961	0.9964	0.9960	0.9961
Final Tuned CNN — Test	0.9953	0.9957	0.9957	0.9957

Table 3: Baseline vs final tuned CNN (best model chosen on validation accuracy).

1. **Final model:** 99.53% test accuracy; 2,308 of 2,319 test images correct; only **11 errors**. Tuning gave a small but consistent gain over the baseline (test 99.35% → 99.53%, about four more images).
2. **Weakest signs** (F1 0.97–0.98): Speed limit 80 and 100 km/h, Dangerous curve right, Children crossing, Right-of-way. Errors are mostly speed-limit mix-ups (for example 50→60, 80→100) and look-alike red warning triangles, as predicted in the domain analysis.

3. **New-image prediction:** the predict_sign() function returns the sign, confidence and top-3 probabilities. Four random test images were all correct (99.9–100% confidence).
4. **Saved model:** TrafficSign_CNN_Model.keras (9.4 MB) plus a class-names JSON; the reloaded model gave identical predictions.

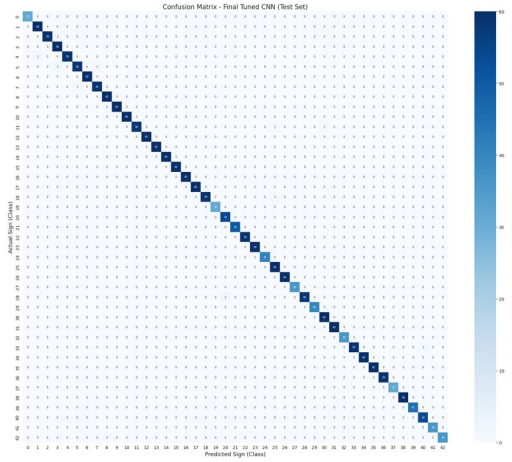


Figure 3: Confusion matrix on the test set (almost perfectly diagonal).

6. Insights, Limitations & Future Scope

Business value: a sign classifier is a building block for driver-assistance alerts and autonomous vehicles; shared preprocessing lets one model handle photos from different cameras; and the confusion analysis shows where to collect more data (speed limits).

Limitations: (a) the random image-level split may put near-identical frames of the same physical sign in train and test, so 99.53% is probably optimistic; (b) only 15,460 of 39,209 images and 64×64 resolution were used; (c) it classifies cropped signs, not signs inside full road scenes; (d) classes follow the German (GTSRB) numbering, so it should not be assumed to work on Indian signs; (e) it is not field-tested and not safety-validated.

Future scope: group-aware split and external validation; use the full dataset on a GPU; higher resolution; transfer learning (MobileNetV2/EfficientNet); a detection stage (YOLO/SSD); real-time camera deployment (TensorFlow Lite); robustness tests for night, rain and glare.

Conclusion: a baseline CNN and a Keras-Tuner-tuned CNN were compared fairly on identical splits; the tuned model reached **99.53% test accuracy (macro F1 0.9957)** and is saved for reuse.

Links: GitHub: https://github.com/Sameekshagithub/PRAICP-1002-Traffic-Sign-Detection-Using-Deep-Learning.git | Drive: https://drive.google.com/drive/folders/1pcADNaZ2l3gGNx7uDhTEBlcqMi5VIEOh?usp=drive_link